

9주차 이 모델을 어떻게 만들었는지 어떻게 증명하는가

9주차 · 실험과 모델 관리

이 장의 수치는 대부분 `practice/chapter9/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

한 가지 예외가 있다. MLflow가 직접 남긴 실험 기록과 모델 등록 결과는 이 강의 환경에서 다시 돌릴 수 없어 **과거 실행이 남긴 산출물에서 옮긴 값**이며, 그 블록의 출처는 `practice/chapter9/data/output/` 으로 적었다.

- ☐ 7주차까지 만든 것은 데이터와 피쳐였고, 이 장에서 처음으로 **모델 훈련**이 등장함
- 훈련이 등장하는 순간 따라오는 질문이 하나 있음 — 그 모델을 어느 데이터로, 어느 설정으로, 어느 코드로 만들었는가
- 이 장은 그 답을 담당자의 기억이 아니라 시스템이 남긴 기록에 두는 방법을 다룸
- ☐ 이 장도 설명과 실습을 나누지 않음
- 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
- 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

- ☐ 석 달 전에 배포한 모델을 지금 다시 만들 수 있는가. 무엇을 알아야 다시 만들 수 있는가.
- ☐ 훈련 한 번에 무엇을 남겨야 "어떻게 만들었는가"에 답이 되는가.
- ☐ 성능 기준을 통과한 새 모델을 운영에 넣지 않는 것이 옳은 경우가 있는가.
- ☐ "지금 운영 중인 모델이 무엇인가"에 사람이 아니라 시스템이 답하게 하려면 무엇이 필요한가.

2. 이 장에서 실행할 것

- ☐ 새 메뉴를 개발하는 식당과 항목이 하나씩 대응함
- 주방에서 하루에도 여러 번 시식하지만, 손님에게 나가는 것은 메뉴판에 오른 것뿐임

식당	이 장
시식 한 번의 기록지 — 재료 배합, 맛 점수, 남겨 둔 레시피 사본	run 하나 — 파라미터, 메트릭, 아티팩트 (artifact)
그날 쓴 재료의 납품 로트 번호	훈련 데이터의 지문(fingerprint) — sha256 해시
메뉴판에 이름과 번호로 올린 정식 메뉴	모델 레지스트리(registry)의 이름과 버전
"오늘의 추천" 팻말 — 손님은 팻말만 보고 주문함	champion 별칭(alias) — 서버가 이것만 바라봄
메뉴판에 올릴지 정하는 심사 기준과 심사자 서명	승격 게이트와 승격 검수 기록

표 9.1 식당의 시식·메뉴판과 이 장의 대응

- ☐ 대응의 핵심은 셋째 줄과 넷째 줄임
- 시식 기록지를 아무리 많이 쌓아도 그것이 곧 메뉴판은 아님 — 실험 기록과 배포 승인은 층이 다름
- 손님이 팻말만 보듯 서버 코드도 버전 번호가 아니라 별칭만 봄
- ☐ 비유가 깨지는 지점이 하나 있음
- 같은 레시피로 두 번 만든 요리는 맛이 조금씩 다르지만, 모델 훈련은 코드·데이터·파라미터·환경이 같으면 결과가 완전히 같아야 함
- 그 동일성을 4절에서 숫자로 확인함
- ☐ 실행하기 전에 네 가지를 먼저 적음
- 남이 만든 산출물을 먼저 보면 거기 적힌 값이 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 9.2 예측표 — 실행하기 전에 채운다

예측할 것	내 예측	실제	어긋났다면 원인
단순한 평균 예측기와 선형회귀 중 훈련 오차가 작은 쪽			
원천 자료 한 칸이 1 바뀌면 지문 앞 16자 중 그대로인 글자 수			
절대 게이트(오차 1.05 이하)만 두면 더 나쁜 후보가 운영에 들어가는가			
승격 검수 기록 13개 점검 항목 중 통과하는 수			

- ☐ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	9-2-training-pairs.py	7주차 확정 집계에서 훈련 쌍과 지문 생성	3	없음
2	9-3-train-and-rerun.py	후보 둘 훈련· 재실행 대조	4	없음
3	9-4-gate-matrix.py	게이트 규칙 셋으로 같은 후보 판정	5	없음
4	9-5-review-record-check.py	승격 검수 기록 점검	6	data/output/ch9_promotion_record.json (저장소에 포함)
선택	9-1-mlflow-tracking.py	같은 절차를 MLflow와 scikit-learn으로	3·5·6	<code>pip install -r code/requirements.txt</code>

□ 1~4번은 표준 라이브러리만 쓰므로 설치할 패키지가 없음(실습 검증 환경은 Python 3.13)

○ 선택 실습만 MLflow 3.14.0과 scikit-learn이 필요함

- 선택 실습은 패키지를 설치한 뒤에 돌린다
- 9-1-mlflow-tracking.py 는 mlflow와 scikit-learn이 있어야 돌아간다. 없으면 아무것도 만들지 못하고 끝난다.
- 실행이 실패해도 다른 실습의 산출물은 남는다. 이 스크립트는 자기가 다시 만드는 실험 기록 (`mlflow.db·mlruns/`)만 비우고, 산출물 JSON은 계산이 끝난 뒤 덮어쓴다.

3. run 하나에 무엇을 남기는가

□ 기록의 단위는 **run**임

○ 훈련을 한 번 돌리면 run 하나가 생기고, **실험(experiment)** 은 그 run들을 묶는 이름임

○ run 하나에 들어가는 것은 세 종류로 나뉨

□ **파라미터(params)**는 **훈련의 입력 설정**임

○ 모델 종류, 하이퍼파라미터(hyperparameter), 표본 수가 여기 들어감

○ "무엇을 넣었는가"에 답하는 항목임

□ **메트릭(metrics)**은 **훈련의 결과 수치**임

○ 이 실습에서는 훈련 MAE(Mean Absolute Error, 평균 절대 오차) 하나임

□ **아티팩트(artifacts)**는 **결과 파일**임

○ 모델 파일, 훈련 데이터 사본처럼 수치로 줄일 수 없는 실물이 들어감

□ 셋을 굳이 나누는 이유는 찾는 방식이 다르기 때문임

- "학습률 0.01로 돌린 run 전부"는 파라미터 검색임
- "오차가 가장 작은 run"은 메트릭 정렬임
- "그 run의 훈련 데이터를 열어 보자"는 아티팩트 내려받기임
- 셋이 한 run에 함께 있어야 기록 단위가 완결됨
- 네 입력 중 셋은 도구가 지켜 주지만 데이터는 아무도 대신 지켜 주지 않음
- 코드는 git이 버전으로 남기고, 파라미터와 메트릭은 기록 도구가 자동으로 남김
- 그런데 훈련 데이터는 **파일 경로가 같아도 내용이 바뀜** — 7주차의 정정 재실행이 그런 경우임
- 그래서 훈련 데이터를 정해진 형식의 텍스트로 만든 뒤 sha256 해시를 찍어 **지문**으로 삼고, 그 지문을 파라미터에 넣음
- 같은 훈련 데이터를 지문과 사본 양쪽으로 남기는 것은 중복이 아니라 용도 분리임
- 지문은 가볍고 비교가 되므로 "같은 데이터였나"에 즉답함
- 사본은 무겁지만 실물이라 "그 데이터를 다시 보자"에 답함
- 첫 번째 코드를 실행함. 7주차 확정 집계에서 훈련 데이터를 만들고 지문을 찍음

```
cd practice/chapter9
python3 code/9-2-training-pairs.py
```

출처: practice/chapter9/results/9-2-training-pairs.log

[1] 원천 - 7주차 확정 집계 3일 × 지역 3개

경로 practice/chapter7/data/input/ch6_daily/

일자	마포구(11440)	관악구(11620)	강남구(11680)
2026-07-01	5	5	8
2026-07-02	5	5	6
2026-07-03	6	5	9

[2] 훈련 쌍 - (전일 건수 → 당일 건수) 6개

지역	전일 → 당일	전일 건수	당일 건수
마포구(11440)	2026-07-01 → 2026-07-02	5	5
마포구(11440)	2026-07-02 → 2026-07-03	5	6
관악구(11620)	2026-07-01 → 2026-07-02	5	5
관악구(11620)	2026-07-02 → 2026-07-03	5	5
강남구(11680)	2026-07-02 → 2026-07-03	6	9
강남구(11680)	2026-07-01 → 2026-07-02	8	6

[3] 정규화 CSV와 지문

첫 줄 lawd_cd,x_prev_count,y_count

행 수 6행 + 머리글 1행

sha256 96f6b6b3ce9dcdd1da62e0284e0ceea1ec2afec24c522f6df3221ad0ab31f49e

9-1 기록값 96f6b6b3ce9dcdd1da62e0284e0ceea1ec2afec24c522f6df3221ad0ab31f49e

일치 여부 예 - 같은 데이터다

[4] 원천 한 칸만 바뀌면 - 마포구 2026-07-03 건수 6 → 7

바뀐 쌍 (5,6) → (5,7) 나머지 5쌍은 그대로

sha256 07060a30f8e15e7b8b4f92e502f8acaf2403f0a85118cb5de2b096b364abaa34

앞에서부터 일치하는 글자 수 0

□ 화면에 나온 값에서 읽을 것

○ **훈련 데이터는 하늘에서 떨어지지 않음** — 6주차 배치가 확정된 지역별 일계를 7주차가 피쳐로 받았고, 이 장은 그 3일치에서 (전일 → 당일) 쌍 6개를 만들었음. 지역마다 이틀분 전이가 나오므로 3지역 × 2 = 6쌍임

○ **지문은 정규화 형식이 정함** — 열 이름·열 순서·행 순서·줄바꿈을 고정한 CSV 한 덩어리에 해시를 찍음. 같은 내용을 다른 순서로 적으면 다른 지문이 나오므로, 지문을 쓰려면 직렬화 규칙부터 고정해야 함

○ **과거 MLflow 실행이 남긴 지문과 같음** — 같은 데이터를 서로 다른 코드로 두 번 요약했는데 64자가 전부 일치함. 이 한 줄이 "9-1이 기록한 그 데이터가 지금 저장소에 있는 그 데이터"라는 증거임

○ 한 칸이 바뀌면 지문 전체가 바뀜 — 6을 7로 고쳤을 뿐인데 앞에서부터 일치하는 글자가 하나도 없음. 지문은 "조금 다르다"를 표현하지 못하고 같은가 다른가만 말함. 예측표에 8자나 12자를 적었다면 여기서 어긋남

□ 선택 실습을 돌리면 같은 값이 MLflow 기록에도 들어감

```
pip install -r code/requirements.txt
python3 code/9-1-mlflow-tracking.py
```

```
with mlflow.start_run(run_name="linear") as run:
    mlflow.log_params({"model_type": "LinearRegression", "data_sha256_16": fp[:16]})
    mlflow.log_metric("train_mae", mae)
    mlflow.log_artifact(str(csv_path), artifact_path="training_data")
    model_info = mlflow.sklearn.log_model(model, name="model")
```

```
# 출처: practice/chapter9/data/output/ch9_experiment_report.json
{
  "data_fingerprint_sha256": "96f6b6b3ce9dcdd1da62e0284e0ceea1ec2afec24c522f6df3221ad0ab31f49e",
  "mlflow_version": "3.14.0",
  "sklearn_version": "1.9.0",
  "runs": {
    "baseline_mean": { "train_mae": 1.0 },
    "linear": { "train_mae": 1.0455, "coef": { "slope": 0.4091, "intercept": 3.6818 } }
  }
}
```

□ 화면에 나온 값에서 읽을 것

○ 파라미터·메트릭·아티팩트를 남기는 API가 각각 따로임 — 코드에서 셋을 구분해 부르는 것이 곧 나중에 셋을 따로 검색하는 근거가 됨

○ 지문은 도구가 자동으로 넣어 주지 않아 log_params 에 직접 실어 보냈음. 자동으로 남는 것과 사람이 설계해 남기는 것의 경계가 여기임

4. 훈련도 함수다 — 같은 입력이면 같은 모델

□ 6주차는 배치를 입력의 결정적 함수로 만들었고, 같은 입력이면 산출물 해시까지 같았음

□ 훈련도 같은 관점으로 봄

○ 모델 = f(코드, 데이터, 파라미터, 환경) 이고, 네 입력이 전부 같으면 같은 모델이 나와야 함

○ 다만 훈련에는 배치에 없던 입력이 하나 숨어 있음 — 초기화·셔플·샘플링이 난수를 쓰는 알고리즘이라면

seed까지 고정해야 함수가 됨

- 이 강의 후보 둘은 난수를 쓰지 않는 결정적 알고리즘이라 seed 없이도 같은 값이 나옴
- 아무것도 기록하지 않으면 재현이 아니라 새 모델이 나옴
- 석 달 전 모델의 성능이 떨어져 "그때 모델을 다시 만들어 비교하자"고 할 때, 데이터·파라미터·코드 중 하나라도 확실하지 않으면 다시 만든 것은 그 모델이 아님
- 비교 실험 자체가 성립하지 않음
- 이것이 게으름의 결과가 아니라는 점이 중요함
- 실험은 하루에 파라미터를 다섯 번 바꾸고 데이터를 두 번 갈아 끼우는 것이 정상임
- 그 속도에서 "나중에 정리하자"는 반드시 실패하므로, 기록은 실험 **후**의 문서화가 아니라 실험 **중**의 자동 기록이어야 함
- 두 번째 코드를 실행함. 후보 둘을 훈련하고 같은 설정으로 한 번 더 훈련함

```
python3 code/9-3-train-and-rerun.py
```

출처: practice/chapter9/results/9-3-train-and-rerun.log

[2] 후보 1 baseline_mean – 입력과 무관하게 y 평균을 내놓는다

예측값 6.0 (= y 합 36 ÷ 6)
잔차 절댓값 [1.0, 0.0, 1.0, 1.0, 3.0, 0.0]
train_mae 1.0000

[3] 후보 2 linear – 전일 건수의 1차 함수

slope 0.4091 (= 9/22)
intercept 3.6818 (= 81/22)
train_mae 1.0455

→ 복잡한 쪽이 기준선보다 나쁘다: 1.0455 > 1.0000

[4] 9-1(scikit-learn)이 남긴 값과 대조

baseline_mean	train_mae	손계산 1.0	기록 1.0	일치 예
linear	train_mae	손계산 1.0455	기록 1.0455	일치 예
linear	slope	손계산 0.4091	기록 0.4091	일치 예
linear	intercept	손계산 3.6818	기록 3.6818	일치 예

[5] 같은 입력으로 다시 훈련(재현성)

지문 96f6b6b3ce9dcdd1... → 96f6b6b3ce9dcdd1...
train_mae 1.0455 → 1.0455 동일 True
계수 (0.4091, 3.6818) → (0.4091, 3.6818) 동일 True

[6] 입력이 한 건 바뀐 채로 같은 코드를 돌리면 – 마포구 마지막 날 건수 +1

지문 96f6b6b3ce9dcdd1... → 07060a30f8e15e7b... 같음 False
baseline_mean train_mae 1.0000 → 1.2222
linear train_mae 1.0455 → 1.2576
linear slope 0.4091 → 0.3182

□ 화면에 나온 값에서 읽을 것

○ 더 복잡한 모델이 기준선에 졌음 — 선형회귀의 오차 1.0455가 평균 예측기의 1.0000보다 큼. 쌍 6개로는 의미 있는 기울기를 배울 수 없으므로 당연한 결과이며, 이 당연함이 5절의 판정을 사실대로 보여 줌

○ 계수가 분수로 떨어짐 — 기울기 9/22, 절편 81/22는 최소제곱 공식을 손으로 푼 값임. 라이브러리가 낸 값과 소수점 넷째 자리까지 같으므로, 모델이 무엇을 계산했는지 학생이 직접 검산할 수 있음

○ 다시 돌려도 같은 값이 나옴 — 지문도 오차도 계수도 동일함. 6주차의 "재실행 산출물 해시 동일"이 여기서 "재훈련 메트릭·계수 동일"이 된 것이며 같은 원리임

○ 입력이 한 건 바뀌면 지문과 메트릭이 함께 움직임 — 지문이 달라졌고 오차는 1.0000에서 1.2222로, 기울기는 0.4091에서 0.3182로 바뀜. 6건 중 1건이 바뀌었을 뿐인데 학습된 함수가 달라짐

- 거꾸로 읽으면 이 확인은 **회귀 검사**가 됨
- "지난달과 같은 설정으로 돌렸는데 오차가 다르다"는 신고가 들어오면, 기록이 있는 한 추리가 아니라 대조 절차가 됨
- 값싼 대조를 먼저, 비싼 대조를 나중에 함
 - 첫째, 두 run의 파라미터를 필드별로 비교함. 다르면 거기서 끝임 — "같은 설정"은 기억이고 파라미터 기록은 사실임
 - 둘째, 파라미터가 전부 같아도 지문이 다르면 데이터가 바뀐 것임. 경로가 같아도 내용은 다름
 - 셋째, 앞의 둘이 같으면 라이브러리 버전을 봄. 수치 계산 라이브러리의 마이너 버전 차이가 미세한 값 차이를 만드는 일은 실재함
 - 넷째, 셋 다 같은데 다르면 고정되지 않은 난수를 찾음
- 이 절차가 성립하려면 첫째·둘째의 기록이 **훈련 시점에** 이미 남아 있어야 함
- 사고가 난 뒤에는 만들 수 없는 기록임
- 정보공개 청구나 감사에서 "이 모델의 판단 근거를 소명하라"는 요구가 왔을 때 모델 파일만 가진 기관이 답하지 못하는 이유도 같음
- 재현성은 연구자의 결벽이 아니라 **책임 소재의 전제 조건**임

5. 승격 게이트는 통과 규칙이 아니라 교체 규칙이다

- 실험을 계속하면 run은 수십, 수백 개가 쌓임
- 그것이 정상이지만, 서빙 시스템에 "run 목록에서 알아서 좋은 것을 골라 쓰라"고 할 수는 없음
- 배포는 다른 규율을 요구함
- 후보는 **이름 있는 모델의 번호 있는 버전**이어야 하고, 지금 운영 중인 것이 어느 버전인지 한 곳에서 답해져야 함
- 그 관문이 **모델 레지스트리**임 — 하나의 이름 아래 배포 후보를 버전으로 관리하는 별도 저장소임
- 각 버전은 자기를 만든 run을 가리키므로 기록과의 연결이 끊기지 않음
- 운영 중인 버전을 가리키는 방법은 **별칭(alias)** 임
- 버전에 champion 같은 가변 이름표를 붙이고, 소비자는 버전 번호가 아니라 역할 이름으로 모델을 불러옴
- 별칭을 v1에서 v3로 옮기는 것이 곧 운영 모델 교체이며, 소비자 코드는 한 줄도 바뀌지 않음
- 7주차에서 서빙이 피쳐를 정의 이름으로 조회했듯, 여기서는 모델을 역할 이름으로 조회함
- 무엇을 통과시킬지 정하는 규칙이 **승격 게이트**임
- 절대 게이트는 "오차가 얼마 이하일 것"처럼 후보 하나만 보고 판정함
- 상대 조건은 "지금 운영 중인 것보다 나을 것"처럼 champion과 비교해 판정함
- 둘은 다른 질문에 답하므로 하나만 두면 각각 다른 사고가 남
- 세 번째 코드를 실행함. 같은 후보에 규칙 셋을 각각 적용해 판정이 갈리는지 봄

python3 code/9-4-gate-matrix.py

출처: practice/chapter9/results/9-4-gate-matrix.log

[1] 후보 3개 - 같은 훈련 쌍 6개, 지문 96f6b6b3ce9dcdd1...

후보	train_mae	입력으로 요구하는 것	설명
baseline_mean	1.0000	없음	훈련 y 전체 평균 - 입력을 보지 않는다
linear	1.0455	전일 건수	전일 건수의 1차 함수
district_mean	0.6667	지역 코드	지역별 y 평균 - 지역마다 다른 값을 낸다

[3] 같은 후보를 세 규칙으로 판정 - champion은 baseline_mean(1.0000)

규칙	linear(1.0455)	district_mean(0.6667)
A 절대만 (mae <= 1.05)	승격	승격
B 상대만 (champion보다 개선)	반려(개선 없음)	승격
C 둘 다	반려(개선 없음)	승격

[4] 규칙 A(절대만)를 쓰면 무엇이 일어나는가

linear 1.0455 <= 1.05 이므로 통과 → champion이 1.0000에서 1.0455로 교체됨
운영 모델의 오차가 커졌는데도 규칙은 정상 작동했다고 기록된다

[5] 임계값을 업무 기준으로 조이면 - mae <= 0.8

baseline_mean	1.0000	미달
linear	1.0455	미달
district_mean	0.6667	통과

등록 순서가 반대여서 linear가 먼저 champion이 되었다면, 상대 조건만으로는
baseline_mean(1.0000 < 1.0455) 판정이 '승격' - 둘 다 업무 기준 미달인데 교체가 일어난다

[6] 별칭 이동과 롤백 - 규칙 C로 district_mean을 v3로 등록한 뒤

champion=v1 강남구 전일 건수 9.0 → 예측 6.00
champion=v3 강남구 전일 건수 9.0 → 예측 7.50
별칭 이력 v1 → v3 → (롤백) v1 소비자 코드는 한 줄도 바뀌지 않는다
반려된 v2를 지우지 않았기 때문에 되돌아갈 수 있는 버전이 v1·v2 둘이다

[7] 지표가 좋아졌다고 끝나지 않는다 - district_mean의 조건

훈련에 있는 지역 11440, 11620, 11680
처음 보는 지역 11110을 넣으면 예측 None

□ 화면에 나온 값에서 읽을 것

○ 절대 게이트만 두면 더 나쁜 모델이 운영에 들어감 — linear는 1.0455로 임계 1.05를 넘지 않아 통과하고, 그대로 champion 1.0000을 밀어냄. 오차가 커졌는데 로그에는 "게이트 통과"만 남음. 예측표 셋째 줄에 "아니오"를 적었다면 여기서 어긋남

○ 상대 조건만 두면 못 쓸 모델끼리 교체가 계속됨 — 업무가 요구하는 오차가 0.8이라면 baseline_mean도

linear도 미달임. 그런데 둘 중 나은 쪽으로 바꾸는 규칙은 계속 작동해, 운영에 쓸 수 없는 모델이 바뀌기만 함

- **둘을 함께 두어야 판정이 업무와 맞음** — 규칙 C에서 linear는 반려되고 district_mean만 승격됨. 절대 게이트가 "쓸 만한가"를 묻고 상대 조건이 "바꿀 이유가 있는가"를 묻음
- **별칭을 옮기면 같은 입력의 답이 바뀜** — 강남구 전일 건수 9.0에 champion v1은 6.00을, v3은 7.50을 냄. 소비자 코드는 그대로인데 결과가 달라지므로, 별칭 이동은 배포 행위이며 이력이 남아야 함
- **반려된 버전을 지우지 않는 것이 롤백 준비임** — 되돌아갈 수 있는 버전이 v1과 v2로 둘임. 승격이 별칭을 앞으로 옮기는 일이면 롤백은 같은 동작의 역방향이므로, 버전을 지우는 순간 복귀 대상이 사라짐

- 오차가 작다고 운영에 맞는 모델은 아니다
- `district_mean` 은 오차가 0.6667로 가장 작지만 예측에 지역 코드를 요구하고, 훈련에 없던 지역 11110을 넣으면 값을 내지 못한다.
- 게다가 지역별 표본이 2개뿐이다. 게이트 지표 하나로 판정하면 이런 조건은 드러나지 않으므로, 승격 규칙에는 지표 외의 요건도 함께 적는다.

□ 9-1을 실행한 환경에서는 같은 판정이 레지스트리에 기록으로 남음

```
# 출처: practice/chapter9/data/output/ch9_promotion_record.json
{
  "model_name": "complaint_daily_forecaster",
  "champion_version": 1,
  "gate_rule": "train_mae <= 1.05 그리고 기존 champion 대비 개선",
  "decisions": [
    { "version": 1, "train_mae": 1.0, "metric_gate_pass": true, "promoted": true,
      "action": "champion 별칭 부여(승격)" },
    { "version": 2, "train_mae": 1.0455, "metric_gate_pass": true, "promoted": false,
      "action": "승격 반려 - 절대 게이트 통과했으나 기존 champion 대비 개선 없음" }
  ]
}
```

□ 화면에 나온 값에서 읽을 것

- v2의 `metric_gate_pass` 가 true 인데 `promoted` 는 false 임 — 절대 게이트 통과와 교체 승인이 서로 다른 칸으로 기록됨
- 두 칸을 하나로 합치면 "왜 통과했는데 안 바꿨는가"라는 감사 질문에 답할 수 없음

6. 승격은 기술 행위가 아니라 승인 행위다

- ☐ 앞의 세 절이 남긴 기록은 "어떻게 만들었고 얼마나 좋은가"에 답함
- ☐ 공공 감사는 거기에 더해 물음
- 누가 만들었고, 무슨 목적으로, 어느 기간의 어떤 출처 데이터로, 어떤 제한 아래 쓰이는가
- ☐ 이 항목들은 성능과 무관하지만 책임과 직결되고, 나중에 소급 작성이 사실상 불가능함
- 담당자는 이동하고 기억은 사라짐
- 그래서 훈련 시점에 run의 태그(tag)로 함께 남김 — 실습이 붙인 최소 집합은 소유자·목적·데이터 출처·데이터 기간 넷임
- ☐ 승격 판정 자체도 문서로 남음
- 공공 운영에서 승격은 승인 행위이므로, 누가 어떤 규칙으로 어떤 근거를 보고 통과시켰는지가 남아야 함
- 이 기록은 사람이 나중에 쓰는 보고서가 아니라 판정 로직이 그 자리에서 낸 출력이어야 함
- ☐ 검수 기록에 재현성 확인이 별도 항목으로 들어가는 이유를 짚어 둠
- 자동 게이트는 수치를 보고, 재현성은 그 수치를 믿어도 되는지를 봄
- **재현되지 않는 run은 게이트 수치가 좋아도 승격 대상이 아님** — 그 수치가 코드와 데이터의 성질인지 우연인지 구분할 수 없기 때문임
- ☐ 네 번째 코드를 실행함. 승격 검수 기록이 감사 질문에 답할 수 있는지 점검함

```
python3 code/9-5-review-record-check.py
```

출처: practice/chapter9/results/9-5-review-record-check.log

[2] 감사 질문별 점검

통과 항목 model_name 어느 모델인가
통과 항목 champion_version 지금 운영 중인 것은 몇 번인가
통과 항목 gate_rule 어떤 규칙으로 통과시켰는가
통과 항목 decisions 각 버전을 어떻게 판정했는가
통과 항목 reproducibility_check 그 수치가 다시 나오는가
통과 항목 data_fingerprint_sha256 어느 데이터로 훈련했는가
통과 항목 reviewer 누가 확인했는가
통과 항목 notes 어떤 제한이 붙는가
통과 지문 재계산 대조 기록된 지문이 원천에서 다시 나오는가
기록 96f6b6b3ce9dcdd1... / 재계산 96f6b6b3ce9dcdd1...
통과 champion 일치 champion 버전이 승격 판정과 맞는가
champion v1 / 승격 판정 [1]
통과 반려 이력 보존 반려된 버전이 기록에 남아 있는가
반려 [2] - 롤백 대상이자 재시도 방지 근거
통과 재현성 통과 같은 설정으로 같은 결과가 나왔는가
mae_equal=True coef_equal=True
미흡 검수자 실명 확인한 사람이 특정되는가
reviewer='검수 담당(가상 - 시뮬레이션 표기)' - 실운영이면 이 칸이 비면 안 된다
→ 12/13 통과

[3] 기록이 부실할 때 - reviewer를 빼고 지문을 다른 값으로 바꾼 사본

미흡 항목 reviewer 누가 확인했는가
미흡 지문 재계산 대조 기록된 지문이 원천에서 다시 나오는가
기록 0000000000000000... / 재계산 96f6b6b3ce9dcdd1...
미흡 검수자 실명 확인한 사람이 특정되는가
→ 10/13 통과

□ 화면에 나온 값에서 읽을 것

- 감사 질문을 점검 항목으로 옮겨 적을 수 있음 — 13개 항목 각각이 "어느 모델인가", "누가 확인했는가" 같은 질문 하나에 대응함. 기록 설계는 감사 질문 목록을 파일 스키마로 옮기는 일임
- 지문 재계산 대조가 기록의 진위를 검증함 — 기록된 64자를 원천 자료에서 다시 계산해 맞춰 봄. 기록만 읽는 점검은 "값이 적혀 있다"까지만 확인하지만, 재계산은 "그 값이 사실이다"까지 확인함
- 12/13에서 걸린 한 칸이 사람 칸임 — 나머지는 자동으로 채워지는데 검수자만 비었음. 교육용이라 가상 표기로 두었지만, 행정 처분이나 자원 배분에 쓰이는 모델이라면 이 칸이 비면 안 됨
- 점검이 실제로 잡아냄 — 검수자를 빼고 지문을 다른 값으로 바꾼 사본은 10/13으로 내려감. 통과 수가 줄어드는 것을 확인해야 그 점검이 형식이 아님을 알 수 있음

- 설계할 때 정할 것이 셋임
- 게이트를 통과했을 때 승격을 자동으로 둘 것인가 사람 확인을 요구할 것인가
- 게이트 미달인데 승격해야 하는 상황에서 누가 예외를 승인하는가
- champion 별칭을 옮길 권한을 누가 갖는가
- 셋째가 특히 실무적임
- 모델을 훈련하는 사람과 별칭을 옮기는 사람이 같으면 게이트는 자기 검사에 그침

7. 핵심 요약

- 1. run 하나는 파라미터·메트릭·아티팩트로 완결되고, 데이터만은 직접 지문을 찍어야 한다. (3절)
 - 입력 설정과 결과 수치와 결과 파일은 찾는 방식이 달라 나누어 남김
 - 코드는 git이, 파라미터는 도구가 지키지만 훈련 데이터는 경로가 같아도 내용이 바뀌므로 sha256 지문을 파라미터로 실어 보냄
 - 원천 한 칸을 6에서 7로 고쳤더니 지문 앞 16자 중 일치하는 글자가 0개였음
- 2. 같은 입력이면 같은 모델이 나오고, 안 나오면 무엇이 바뀌었는지 기록이 갈라 준다. (4절)
 - 선형회귀를 다시 훈련해도 오차 1.0455와 계수 (0.4091, 3.6818)가 그대로였음
 - 입력을 한 건만 바꾸자 지문이 달라지고 오차가 1.0000에서 1.2222로 움직였음
 - 파라미터 → 지문 → 환경 → 난수 순으로 대조하되, 앞의 둘은 훈련 시점에 남겨야만 존재함
- 3. 절대 게이트와 상대 조건은 다른 질문에 답하므로 둘 다 필요하다. (5절)
 - 절대 게이트만 두면 1.0455가 임계 1.05를 통과해 1.0000인 현행을 밀어냄
 - 상대 조건만 두면 업무 기준 0.8에 둘 다 미달인 후보끼리 교체가 계속됨
 - 둘을 함께 둔 규칙 C에서만 linear가 반려되고 district_mean이 승격됐음
- 4. 별칭 이동이 배포이고, 반려 버전을 남기는 것이 롤백 준비다. (5절)
 - 같은 입력 9.0에 champion v1은 6.00을, v3은 7.50을 냈고 소비자 코드는 바뀌지 않았음
 - 반려된 v2를 지우지 않았기에 되돌아갈 수 있는 버전이 둘로 남았음
- 5. 승격은 승인 행위이고, 검수 기록은 감사 질문 목록을 파일로 옮긴 것이다. (6절)
 - 13개 점검 중 12개가 자동으로 통과했고 걸린 한 칸이 검수자였음
 - 지문을 원천에서 다시 계산해 맞춰야 기록의 진위까지 확인됨
 - 훈련하는 사람과 champion 별칭을 옮기는 사람이 같으면 게이트는 자기 검사에 그침

8. 과제

- 실습을 순서대로 실행해 만든 산출물을 LMS에 제출한다
 1. 표 9.2 예측표 — 실행 결과와 대조해 채운 최종본. 텍스트든 이미지든 형식은 무방하다.
 2. `practice/chapter9/data/output/ch9_training_runs.json`
 3. `practice/chapter9/data/output/ch9_gate_matrix.json`
 4. `practice/chapter9/data/output/ch9_record_audit.json`

5. `practice/chapter9/data/output/ch9_experiment_report.json`

☐ 5번은 선택 실습(9-1-mlflow-tracking.py)의 산출물이다

○ MLflow 설치가 막혀 실행하지 못했으면 저장소에 포함된 파일을 그대로 내고, 어디서 막혔는지 한 줄로 적는다

☐ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. `ch9_training_runs.json`의 `data_fingerprint_sha256` 과

`input_changed_one_cell.data_fingerprint_sha256` 을 나란히 옮겨 적고, 훈련 데이터가 바뀐 것을 지문 없이 알아내려면 무엇이 필요한지 쓴다.

2. `ch9_gate_matrix.json`의 `gate_matrix` 에서 규칙 A와 규칙 C의 `linear` 판정을 옮겨 적고, 두 판정이 같린 이유를 쓴다.

3. `ch9_gate_matrix.json`의 `alias_move` 에서 `forecast_champion_v1` 과 `forecast_champion_v3` 를 옮겨 적고, 별칭을 옮길 때 소비자에게 무엇을 알려야 하는지 쓴다.

4. `ch9_record_audit.json` 에서 `pass` 가 `false` 인 항목을 찾아 이름을 적고, 그 칸을 채우려면 누가 무엇을 해야 하는지 쓴다.

☐ 값을 지어내지 않는다

☐ 실행이 안 되면 어디서 막혔는지 그대로 쓴다

☐ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

☐ 실습을 더 해 보려면 `practice/chapter9/` 에서 다음을 시도함

○ 9-4-gate-matrix.py 의 `GATE_BUSINESS` 값을 바꿔, 임계를 몇으로 두면 어떤 후보도 통과하지 못하는지 찾을

○ 9-2-training-pairs.py 의 정규화 규칙(열 순서·정렬 기준)을 바꿔, 내용이 같은데도 지문이 달라지는 것을 확인함

○ 9-5-review-record-check.py 의 `REQUIRED` 표에 항목을 더해, 자기 기관이라면 어떤 질문을 더 물을지 점검 항목으로 옮김

○ MLflow를 설치했다면 `mlflow ui --backend-store-uri sqlite:///data/output/mlflow.db` 로 run 목록과 아티팩트를 화면에서 봄

☐ 뒤 주차에서 이어지는 내용

○ 승격된 모델을 실제 서비스로 내보내고 되돌리는 절차는 10주차, 배포 후 성능이 조용히 나빠지는 것을 잡는 일은 11주차에서 실행함

○ 피처 수준의 권한·이력 관리는 12주차, 이 장의 기록을 이용자에게 설명하는 문제는 14주차에서 이어짐

☐ 원문을 직접 보고 싶다면 다음 두 편이 출발점임

○ MLflow Documentation, 「Model Registry」 — 버전·별칭·단계의 정의. <https://mlflow.org/docs/latest/ml/>

[model-registry/](#)

○ Mitchell 외, 「Model Cards for Model Reporting」 — 모델을 외부에 설명하는 공개 양식. <https://arxiv.org/abs/1810.03993>